ESTÁCIO

Trabalho Prático

Desenvolvimento de Aplicativos Móveis com Flutter

DGT2812 - Posso criar um App de outra forma!

App Explore Mundo

Aluno: Gabriel Cosso

Repositório: https://github.com/biel-cosso/explore-mundo-flutter

Site: https://biel-cosso.github.io/explore-mundo-flutter/

Data: outubro de 2026

1. Objetivo

A agência de viagens Explore Mundo quer um aplicativo onde o cliente possa ver os destinos, consultar pacotes, fazer reserva, falar com a equipe e ver a localização e as avaliações em estrelas de cada lugar. Os pedidos principais eram um banner de destaque com fotos clicáveis, uma barra de navegação (destinos, pacotes de viagem, contato e sobre nós) e uma pesquisa rápida. A agência também quer um site.

Fiz o app em Flutter. Como o Flutter gera Android e web com o mesmo código, o site complementar é a versão web do próprio app, publicada no GitHub Pages.

2. Microatividades

Microatividade 1: preparação do ambiente

O Git eu já tinha. Baixei o Flutter SDK 3.47.8, extraí em `C:\flutter` e coloquei `C:\flutter\bin` no Path do usuário, como o roteiro explica. O Android Studio já estava instalado do outro trabalho. Faltava o Android SDK 36, que o `flutter doctor` apontou, e instalei. Para testar criei um emulador Pixel 7 com Android 15 (API 35). Resultado do `flutter doctor`:

```
[√] Flutter (Channel stable, 3.47.8, on Microsoft Windows, locale pt-BR)
[√] Windows Version (Windows 11 or higher, 25H2)
[√] Android toolchain - develop for Android devices
[√] Chrome - develop for the web
[X] Visual Studio - develop Windows apps
[√] Connected device
[√] Network resources
```

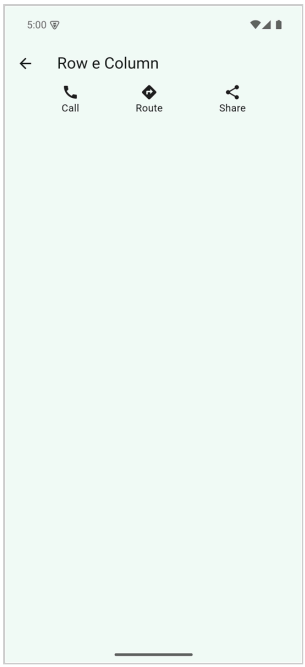
O item do Visual Studio só é necessário para fazer apps de computador Windows, que não é o caso deste trabalho.

Microatividades 2 a 5

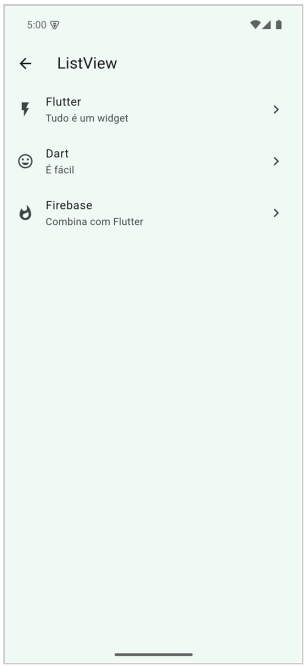
Coloquei cada microatividade numa tela separada dentro do app, que abre pelo menu lateral (ícone ☰). Usei os códigos do roteiro: o `StatelessWidgetExemplo` com MaterialApp, Scaffold e AppBar; a Row com três Columns (Call, Route e Share); a ListView com os ListTile de Flutter, Dart e Firebase; e o Stack com os três Containers coloridos.



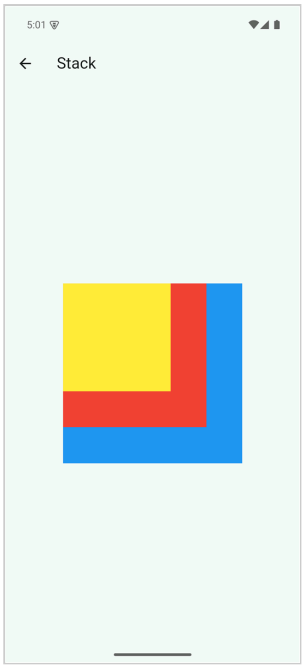
Microatividade 2



Microatividade 3



Microatividade 4



Microatividade 5

Os mesmos widgets também foram usados no app em si. O Stack coloca o nome do destino por cima da foto no banner, a Row com Columns é a base dos botões LIGAR/ROTA/COMPARTILHAR, e a ListView com ListTile monta as listas de destinos, pacotes e avaliações.

3. O aplicativo

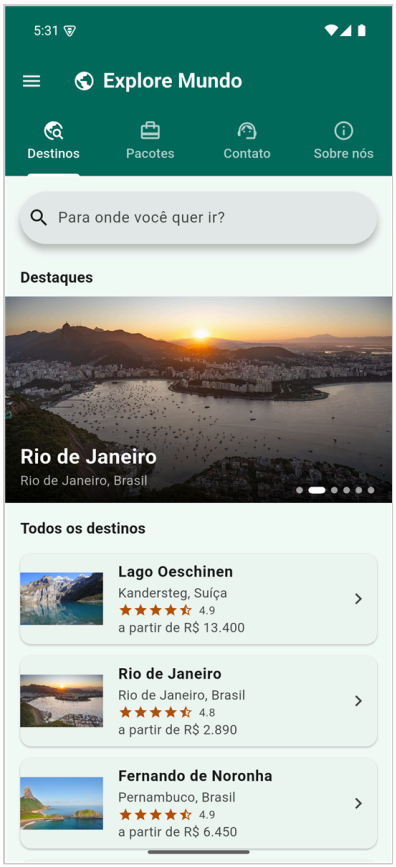


Figura 1 - Tela inicial com as abas, a pesquisa, o banner e os destinos

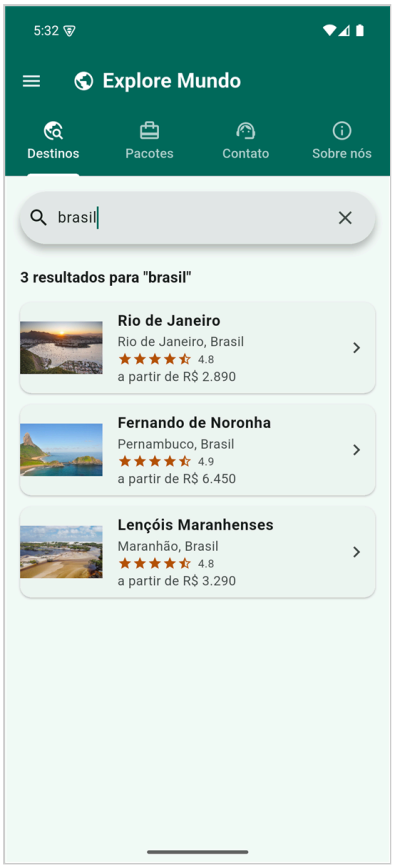


Figura 2 - Pesquisa rápida

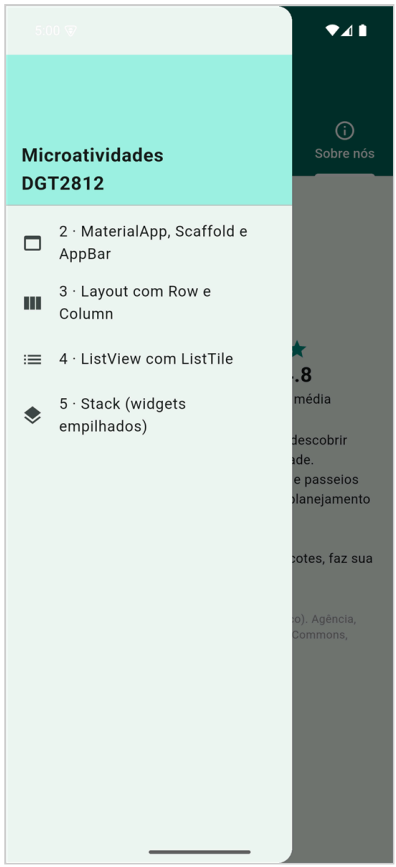


Figura 3 - Menu lateral

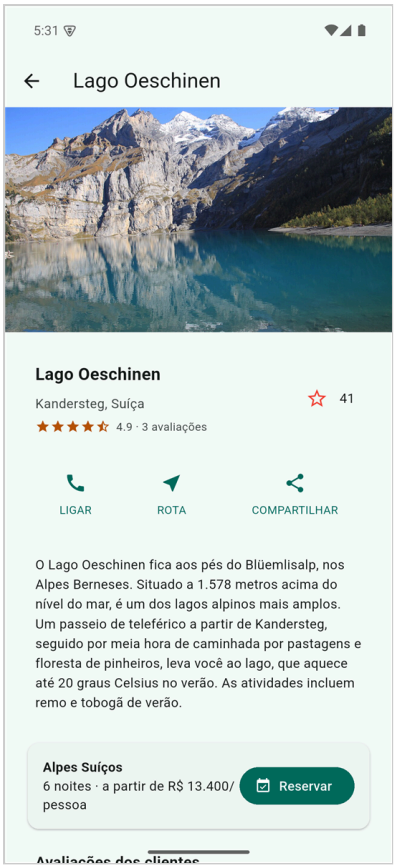


Figura 4 - Página do destino

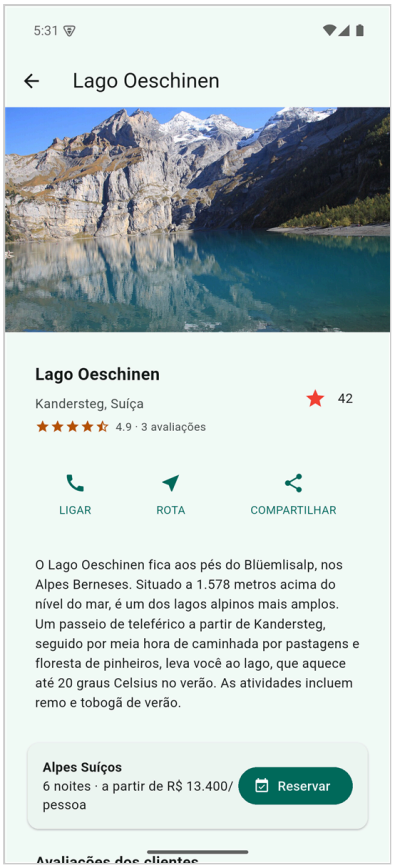


Figura 5 - Favorito marcado (41 para 42)

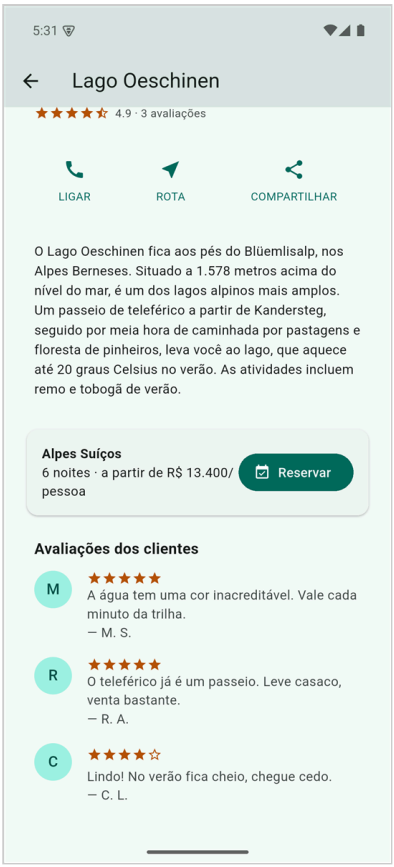


Figura 6 - Pacote e avaliações

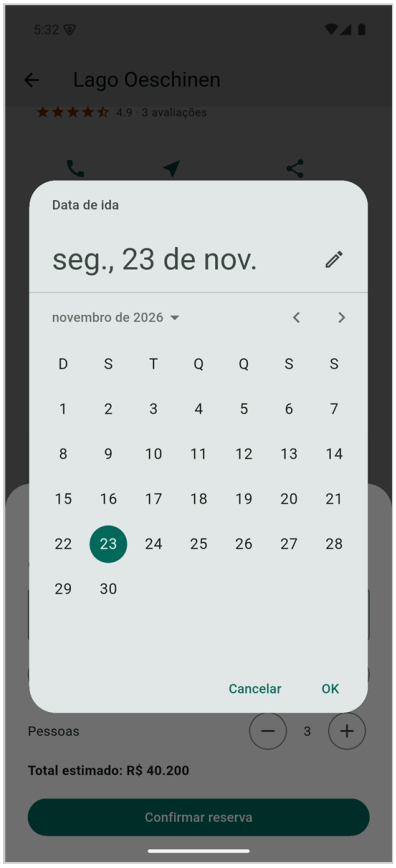


Figura 7 - Escolha da data

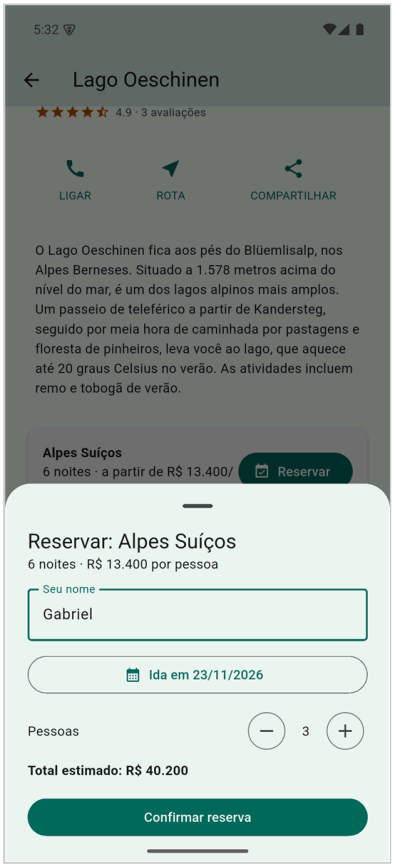


Figura 8 - Reserva preenchida

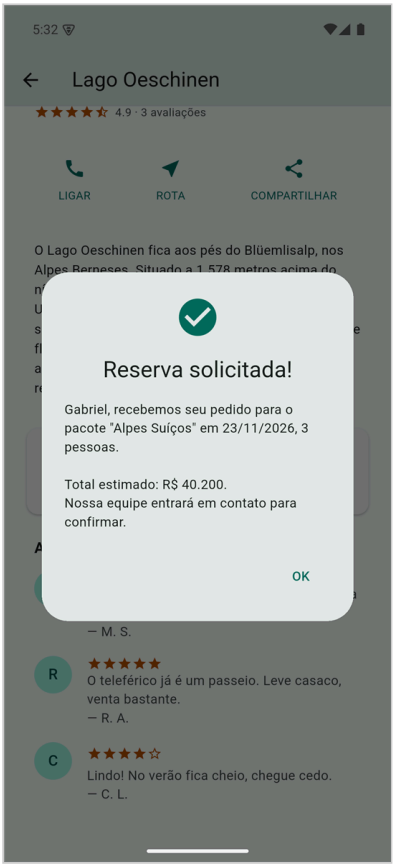


Figura 9 - Reserva solicitada

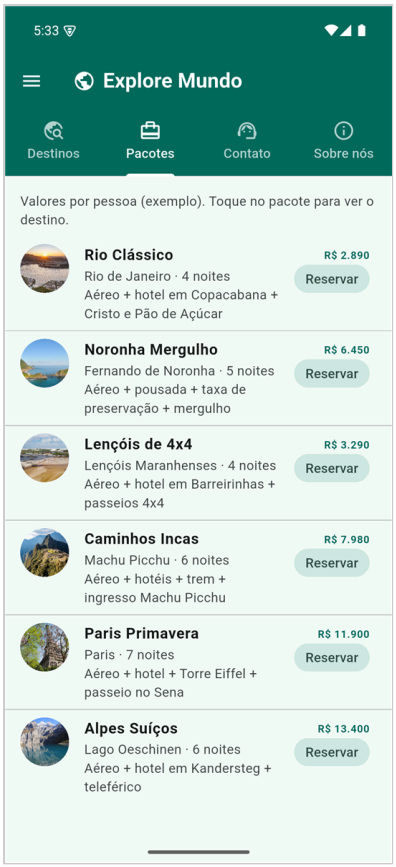


Figura 10 - Pacotes

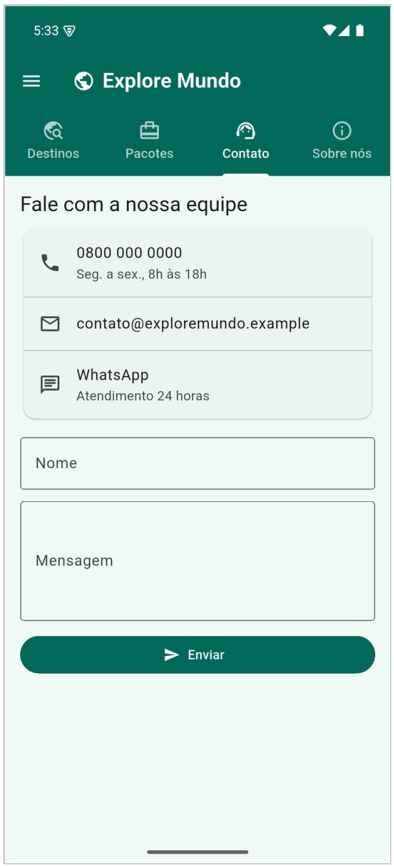


Figura 11 - Contato



Figura 12 - Sobre nós

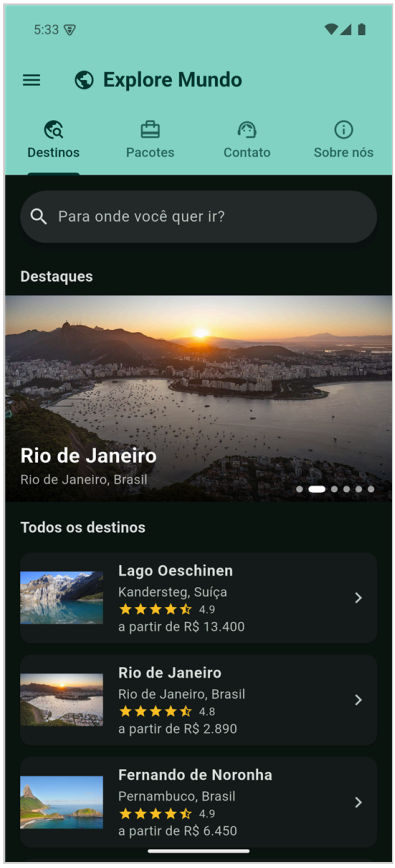


Figura 13 - Tema escuro

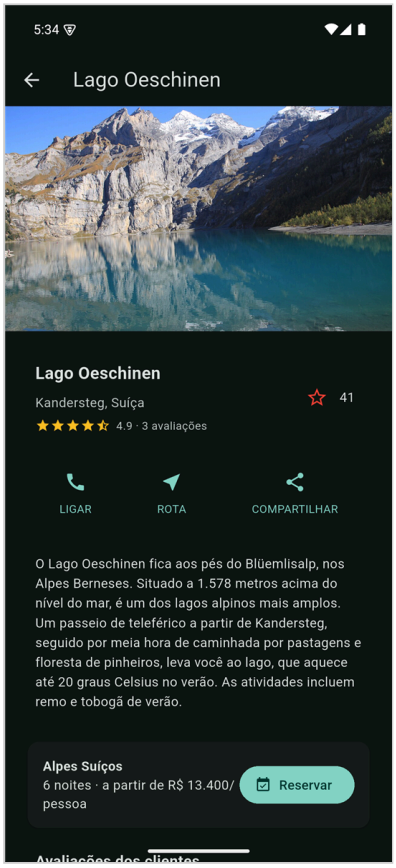


Figura 14 - Destino no tema escuro

4. Como fiz o roteiro

Criei o projeto com `flutter create` e comecei pelo `main.dart` do passo 2. Para a página do destino fiz como os passos 3 e 4 mandam: dividi a tela em imagem, seção de título, seção de botões e texto, e guardei cada parte numa variável para o código não ficar todo aninhado. Isso está em `lib/telas/destino.dart`.

Passo 5: seção de título

```
Widget titleSection = Container(
  padding: const EdgeInsets.all(32),
  child: Row(children: [
    Expanded(child: Column(crossAxisAlignment: CrossAxisAlignment.start, children: [
      Container(padding: const EdgeInsets.only(bottom: 8),
                child: Text(d.nome, style: const TextStyle(fontWeight: FontWeight.bold))),
      Text(d.local, style: TextStyle(color: cores.onSurfaceVariant)),
      Estrelas(nota: d.nota, rotulo: '4.9 · 3 avaliações'),
    ])),
    FavoritoWidget(totalInicial: d.favoritos),
  ]),
);
```

No roteiro a estrela vermelha e o número 41 são fixos. Para a funcionalidade personalizada, transformei isso no `FavoritoWidget`, um StatefulWidget: tocar na estrela marca o favorito e soma 1, tocar de novo desfaz.

Passo 6: seção de botões

```
Widget buttonSection = Row(
  mainAxisAlignment: MainAxisAlignment.spaceEvenly,
  children: [
    _buildButtonColumn(color, Icons.call, 'LIGAR', () => _ligar(context)),
    _buildButtonColumn(color, Icons.near_me, 'ROTA', () => _rota(context)),
    _buildButtonColumn(color, Icons.share, 'COMPARTILHAR', () => _compartilhar(context)),
  ],
);
```

O `_buildButtonColumn` é o do roteiro, mas coloquei um `InkWell` em volta para os botões funcionarem: LIGAR abre o discador, ROTA abre o Google Maps nas coordenadas do destino (pacote `url_launcher`) e COMPARTILHAR copia um texto sobre o destino.

Passos 7, 8 e 9: texto, imagem e ListView

```
Widget textSection = Container(padding: const EdgeInsets.all(32),
                               child: Text(d.descricao, softWrap: true));

body: ListView(children: [
  Image.asset(d.imagem, width: 600, height: 240, fit: BoxFit.cover),
  titleSection, buttonSection, textSection,
]),
```

As seis fotos estão na pasta `images` e declaradas no `pubspec.yaml`, como no passo 8. A ListView do passo 9 faz a página rolar no celular. Abaixo do texto coloquei o cartão do pacote com o botão Reservar e as avaliações.

Banner, navegação e pesquisa

O banner é um `PageView` com um `Timer` que passa para a próxima foto a cada 4 segundos. A barra de navegação é uma `TabBar` dentro da AppBar. A pesquisa usa o `SearchBar` e filtra a lista enquanto a pessoa digita, procurando no nome, no país e na descrição, sem ligar para acento ("lencois" encontra "Lençóis").

RaisedButton

Os objetivos falam em RaisedButton, mas esse widget foi removido do Flutter. Usei os botões que substituem ele: `FilledButton` para as ações principais (Reservar, Confirmar reserva e Enviar) e `ElevatedButton`/ `OutlinedButton` nos outros.

5. Site complementar

Gerei a versão web com `flutter build web` e publiquei no GitHub Pages. No computador o conteúdo fica centralizado com largura máxima de 900 px, para não esticar a tela toda.

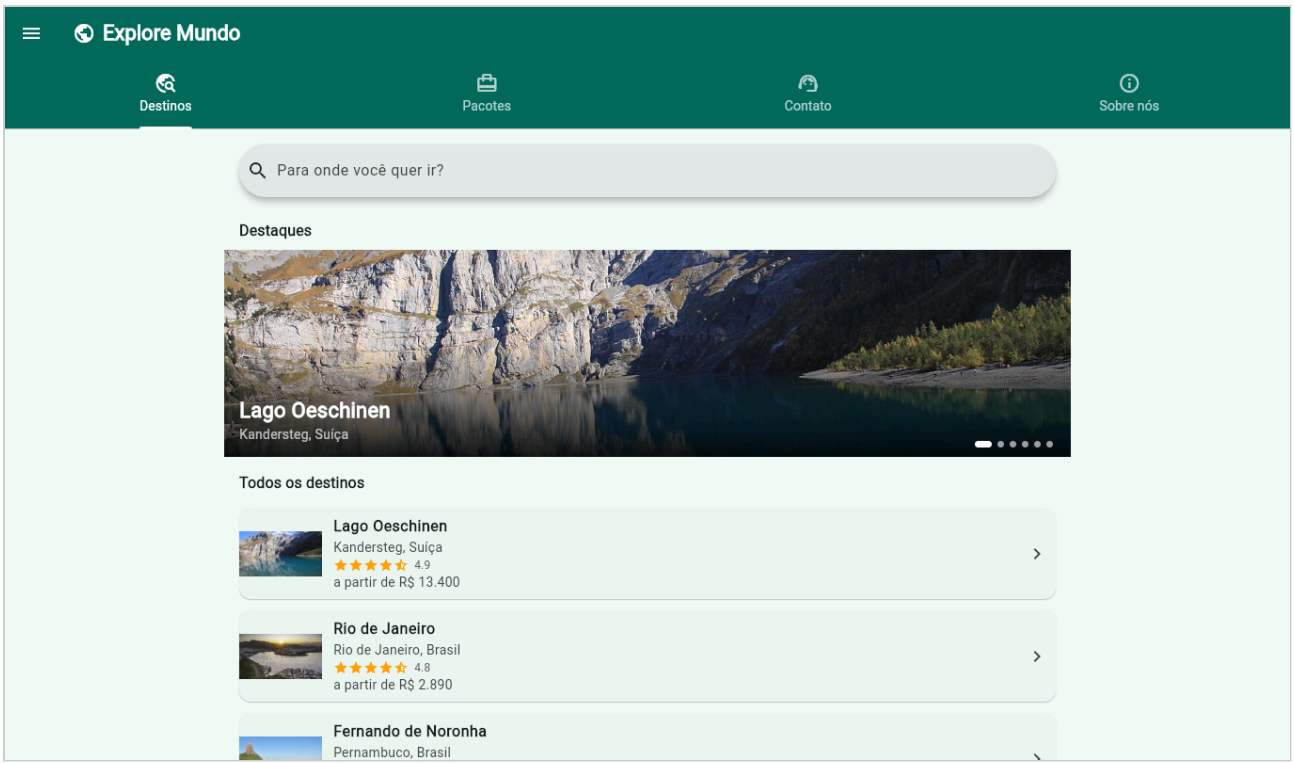


Figura 15 - Site no navegador.

6. Testes

Além de testar no emulador, escrevi testes automáticos (pasta `test`, rodam com `flutter test`). Os 19 testes passaram:

- pesquisa (com e sem acento, sem resultado), formatação de preço e plural das palavras;
- favorito somando e desfazendo;
- reserva não confirma sem nome e sem data, e o total muda junto com o número de pessoas;
- contraste das cores pela fórmula da WCAG (mínimo 4,5:1 para texto), nos temas claro e escuro.

O teste de contraste reprovou duas cores que eu tinha usado no começo: o cinza dos textos pequenos (2,6:1) e o amarelo das estrelas (1,9:1). Troquei pelas cores do tema e por um laranja mais escuro. Agora o texto secundário fica com 8,9:1 e as estrelas com 4,8:1. O texto branco do banner também é testado no pior caso, com uma foto toda branca atrás.

7. Dificuldades que tive

- A primeira compilação para Android demorou uns 15 minutos, porque o Flutter baixou o NDK e outras peças.
- O emulador do celular ficou travando ("System UI isn't responding") nos primeiros minutos, com o computador carregado. Esperando o sistema terminar de iniciar, ele estabilizou.
- O calendário da reserva aparecia em inglês. Resolvi com o pacote `flutter_localizations`, colocando o app em português do Brasil.
- No site, o conteúdo ficava esticado na tela inteira do computador. Limitei a largura.

8. Observações

A agência, os preços, as avaliações e os contatos são inventados para o trabalho, e a reserva e o contato não enviam dados para lugar nenhum. As descrições dos destinos são informações reais. As fotos são do Wikimedia Commons com licenças livres:

Foto	**Autor**	**Licença**
Lago Oeschinen (Oeschinensee 0437)	Martin Thurnherr	CC BY-SA 4.0
Rio de Janeiro visto do Pão de Açúcar	Wilfredor	CC0
Fernando de Noronha, Mirante do Forte dos Remédios	Brunno Monteiro Lira	CC BY-SA 3.0
Paris, Torre Eiffel na primavera	Jorge Royan	CC BY-SA 3.0
Machu Picchu	Diego Delso	CC BY-SA 4.0
Lençóis Maranhenses	Matheus Hobold Sovernigo	CC BY-SA 4.0

9. Conclusão

O trabalho juntou tudo que foi visto nas microatividades: MaterialApp, Scaffold, AppBar, Text, botões, Row, Column, Stack, ListView e ListTile, e um widget com comportamento próprio. Com eles atendi o que a agência pediu (banner, navegação, pesquisa, avaliações, pacotes, reserva e contato). O que mais me chamou atenção foi conseguir entregar o app Android e o site com o mesmo código.